PROBLEM 8 · SOLUTION

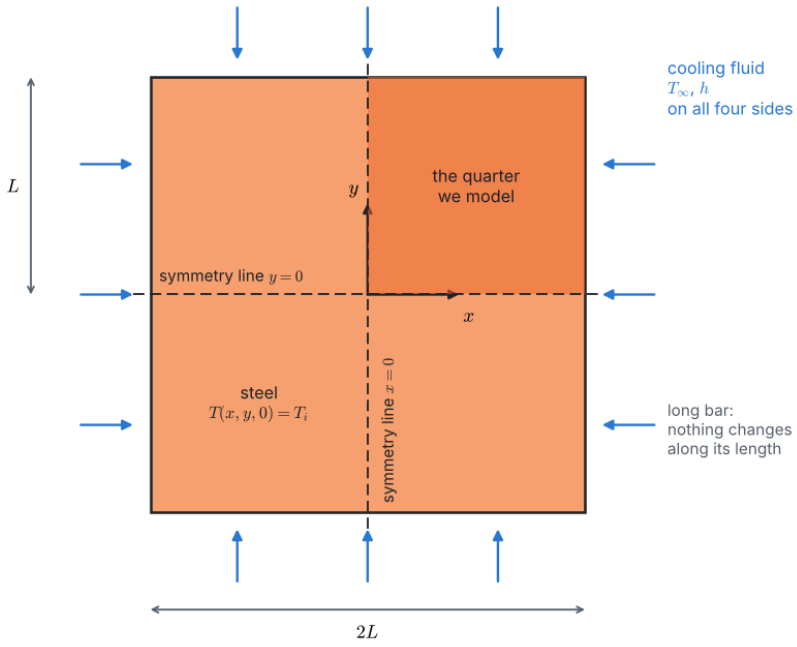
Cooling a steel billet

Step-by-step solution with the Finite Difference Method

Plan. We follow the 7 pillars of CFD: geometry → grid → physics → numerical scheme → solution setup → compute → post-processing. All code below is plain Python (lists and loops only; `math`, `time` and `json` from Python itself). It is the code of the notebook `03_fdm_solver.ipynb`, where it can be run cell by cell.

Step 1 · Geometry, data, and the lumped-capacitance check

A 100 mm × 100 mm steel billet at 600 °C is cooled by a water-mist spray (30 °C, $h = 1000$ W/(m²·K)) on all four sides. The properties are those of plain carbon steel at 600 K (Incropera et al., Table A.1), taken as constant.



The billet cross-section and the quarter we model.

```
# the billet
L = 0.050            # half-width of the square cross-section [m] (the billet is 2L = 100 mm wide)
T_init = 600.0       # initial temperature [°C]
T_inf = 30.0         # temperature of the cooling fluid [°C]
h = 1000.0           # convection coefficient [W/(m^2 K)]
T_target = 100.0     # we want the time at which the centre reaches this temperature [°C]

# plain carbon steel at 600 K (Incropera et al., Table A.1)
k = 48.0             # thermal conductivity [W/(m K)]
rho = 7854.0         # density [kg/m^3]
c_p = 559.0          # specific heat [J/(kg K)]

alpha = k / (rho * c_p)          # thermal diffusivity [m^2/s]
Bi = h * L / k                   # Biot number based on the half-width

print(f"thermal diffusivity alpha = {alpha:.4e} m^2/s")
print(f"diffusion time L^2/alpha  = {L**2 / alpha:.1f} s")
print(f"Biot number Bi = h L / k  = {Bi:.4f}")
```

```
thermal diffusivity alpha = 1.0933e-05 m^2/s
diffusion time L^2/alpha  = 228.7 s
Biot number Bi = h L / k  = 1.0417
```

Could one temperature be enough? Lumped capacitance treats the whole billet as one temperature, $T(t) = T_\infty + (T_i - T_\infty)\, e^{-t/\tau}$ with $\tau = \rho c_p V/(hA)$. It is valid only if $Bi = hL_c/k < 0.1$, with $L_c = V/A = L/2$ for a square bar:

```
L_c = (2 * L) ** 2 / (4 * 2 * L)           # volume / surface area, per unit length [m]
Bi_lumped = h * L_c / k
tau = rho * c_p * L_c / h                  # time constant of the lumped model [s]
t_lumped = tau * math.log((T_init - T_inf) / (T_target - T_inf))

print(f"L_c = V/A = {L_c * 1000:.1f} mm,  Bi = h L_c / k = {Bi_lumped:.3f}  (must be < 0.1)")
print(f"lumped capacitance: tau = {tau:.1f} s, {T_target:g} °C after {t_lumped:.1f} s")
```

```
L_c = V/A = 25.0 mm,  Bi = h L_c / k = 0.521  (must be < 0.1)
lumped capacitance: tau = 109.8 s, 100 °C after 230.2 s
```

$Bi \approx 0.52$ is five times too large: the surface cools much faster than the centre. We need the temperature field $T(x, y, t)$.

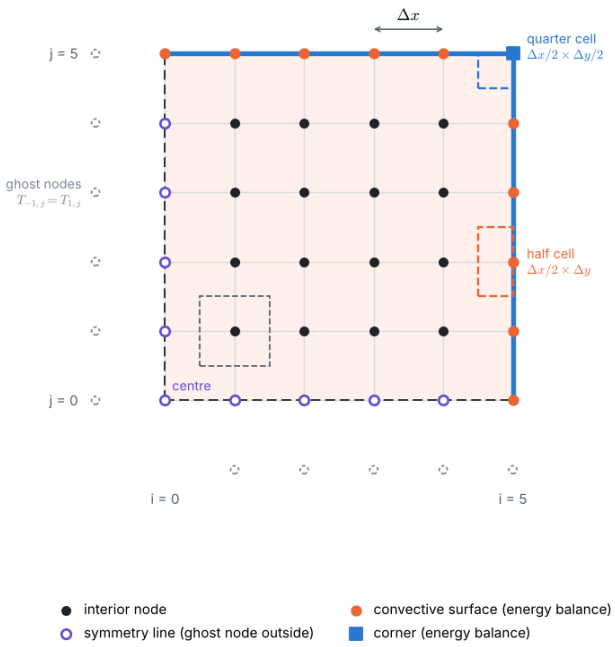
Step 2 · Symmetry and the grid

The billet, its initial temperature and its cooling are symmetric about the lines $x = 0$ and $y = 0$ through its centre, so no heat crosses them. We compute **one quarter**, $0 \le x, y \le L$, with $N = 10$ intervals per side. Every node is unknown, including the surface nodes: their temperature follows from the convection condition.

```
N = 10                     # intervals per side of the quarter
dx = L / N                 # grid spacing [m] (dy = dx)
x = [i * dx for i in range(N + 1)]
Bi_cell = h * dx / k       # Biot number of one grid cell, appears in the surface equations

print(f"N = {N}: dx = {dx * 1000:.1f} mm, {(N + 1) ** 2} nodes (all unknown)")
print(f"Bi_cell = h dx / k = {Bi_cell:.5f}")
```

```
N = 10: dx = 5.0 mm, 121 nodes (all unknown)
Bi_cell = h dx / k = 0.10417
```



The quarter grid (drawn with $N = 5$): interior nodes, symmetry-line nodes with ghost nodes outside, convective-surface nodes (half control volumes) and the corner (quarter control volume).

Step 3 · Physics and the exact solution

Inside the steel, $\partial T/\partial t = \alpha\,(\partial^2 T/\partial x^2 + \partial^2 T/\partial y^2)$, with $T = T_i$ at $t = 0$, $\partial T/\partial n = 0$ on the symmetry lines and $-k\,\partial T/\partial n = h\,(T - T_\infty)$ on the two cooled surfaces.

Exact solution. For a plane wall of half-thickness L with the same conditions, $\theta^* = (T - T_\infty)/(T_i - T_\infty)$ is

$$P(x,t) = \sum_{n=1}^{\infty} C_n\, e^{-\zeta_n^2 Fo} \cos\!\left(\zeta_n \frac{x}{L}\right), \qquad Fo = \frac{\alpha t}{L^2}, \qquad C_n = \frac{4\sin\zeta_n}{2\zeta_n + \sin 2\zeta_n}, \qquad \zeta_n \tan\zeta_n = Bi.$$

The square bar is the intersection of two such walls, and its solution is their **product** (Incropera et al., §5.8):

$$\frac{T(x,y,t) - T_\infty}{T_i - T_\infty} = P(x,t)\,P(y,t).$$

```python
def eigenvalues(Bi, n_terms):
    """The first n_terms positive roots of zeta tan(zeta) = Bi, by bisection."""
    roots = []
    for n in range(n_terms):
        low, high = n * math.pi, n * math.pi + math.pi / 2       # one root in each interval
        for _ in range(60):                                      # 60 halvings: 1e-18 of the interval
            mid = (low + high) / 2
            if mid * math.tan(mid) > Bi:
                high = mid
            else:
                low = mid
        roots.append((low + high) / 2)
    return roots


zeta = eigenvalues(Bi, 50)
C_n = [4 * math.sin(z) / (2 * z + math.sin(2 * z)) for z in zeta]


def plane_wall(x, t):
    """Dimensionless temperature of a plane wall, P(x, t) (x measured from the centre)."""
    Fo = alpha * t / L**2
    return sum(c * math.exp(-z * z * Fo) * math.cos(z * x / L) for z, c in zip(zeta, C_n))


def T_exact(x, y, t):
    """Exact temperature in the billet: the product of two plane walls."""
    if t == 0:
        return T_init
    return T_inf + (T_init - T_inf) * plane_wall(x, t) * plane_wall(y, t)


for n in range(3):
    check = zeta[n] * math.tan(zeta[n])
    print(f"zeta_{n + 1} = {zeta[n]:.6f}  (zeta tan zeta = {check:.6f})   C_{n + 1} = {C_n[n]:+.6f}")
```

```
zeta_1 = 0.873168  (zeta tan zeta = 1.041667)   C_1 = +1.122485
zeta_2 = 3.435952  (zeta tan zeta = 1.041667)   C_2 = -0.156251
zeta_3 = 6.443461  (zeta tan zeta = 1.041667)   C_3 = +0.048353
```

```python
def exact_time_to(T_goal, x=0.0, y=0.0):
    """Time at which the exact temperature at (x, y) falls to T_goal (bisection)."""
    low, high = 0.1, 10_000.0
    for _ in range(80):
        mid = (low + high) / 2
        if T_exact(x, y, mid) > T_goal:
            low = mid
        else:
            high = mid
    return (low + high) / 2


t_exact = exact_time_to(T_target)
print(f"exact: the centre reaches {T_target:g} °C after {t_exact:.1f} s ({t_exact / 60:.2f} min)")
print(f"       at that moment the middle of a face is at {T_exact(L, 0, t_exact):.1f} °C "
      f"and the corner at {T_exact(L, L, t_exact):.1f} °C")
print(f"lumped capacitance: {t_lumped:.1f} s, {100 * (t_lumped - t_exact) / t_exact:+.0f} %")
```

```
exact: the centre reaches 100 °C after 349.1 s (5.82 min)
       at that moment the middle of a face is at 75.0 °C and the corner at 58.9 °C
lumped capacitance: 230.2 s, -34 %
```

Step 4 · Numerical scheme

Explicit FTCS from energy balances. For each control volume, (heat stored per time step) = (heat flowing in at the old time level). With $r = \alpha\,\Delta t/\Delta x^2$ and $Bi_{cell} = h\,\Delta x/k$:

node (control volume)	FTCS update
interior (full cell)	$T_P + r\,(T_E + T_W + T_N + T_S - 4T_P)$
symmetry line $x = 0$ (ghost $T_W = T_E$)	$T_P + r\,(2T_E + T_N + T_S - 4T_P)$
convective surface $x = L$ (half cell)	$T_P + r\,(2T_W + T_N + T_S - 4T_P) + 2\,r\,Bi_{cell}\,(T_\infty - T_P)$
corner (quarter cell)	$T_P + 2\,r\,(T_W + T_S - 2T_P) + 4\,r\,Bi_{cell}\,(T_\infty - T_P)$

For example, the half cell of a surface node, $\frac{\Delta x}{2} \times \Delta y$, receives heat from W through a face Δy long, from N and S through faces $\Delta x/2$ long, and from the fluid through the surface:

$$\rho c_p \frac{\Delta x\,\Delta y}{2}\,\frac{T_P^{n+1} - T_P}{\Delta t} = k\,\Delta y\,\frac{T_W - T_P}{\Delta x} + k\,\frac{\Delta x}{2}\,\frac{T_N - T_P}{\Delta y} + k\,\frac{\Delta x}{2}\,\frac{T_S - T_P}{\Delta y} + h\,\Delta y\,(T_\infty - T_P).$$

Every row of the table is the sum of an x part and a y part, each a 1D second difference with a ghost node: $2(T_1 - T_0)$ on a symmetry line, $T_{i+1} - 2T_i + T_{i-1}$ inside, and $2(T_{N-1} - T_N) - 2Bi_{cell}T_N + 2Bi_{cell}T_\infty$ on a cooled surface (the convection condition fixes the ghost value T_{N+1}). So one function serves all nodes:

```python
def second_difference(v, i, N, b):
    """1D second difference at position i of a row or column v, with ghost nodes:
    symmetry at i = 0, convection at i = N (b = Bi_cell; the T_inf part is added separately)."""
    if i == 0:
        return 2 * (v[1] - v[0])
    if i == N:
        return 2 * (v[N - 1] - v[N]) - 2 * b * v[N]
    return v[i + 1] - 2 * v[i] + v[i - 1]


def ftcs_step(T, N, r, b):
    """One explicit (FTCS) time step on the quarter billet; returns a new grid."""
    columns = [[T[j][i] for j in range(N + 1)] for i in range(N + 1)]
    new = [[0.0] * (N + 1) for _ in range(N + 1)]
    for j in range(N + 1):
        for i in range(N + 1):
            change = second_difference(T[j], i, N, b) + second_difference(columns[i], j, N, b)
            if i == N:
                change += 2 * b * T_inf          # convection on the surface x = L
            if j == N:
                change += 2 * b * T_inf          # convection on the surface y = L
            new[j][i] = T[j][i] + r * change
    return new
```

Stability. The weight of T_P in each update must not be negative:

node	weight of T_P	positive if
interior, symmetry, centre	$1 - 4r$	$r \le 1/4$
convective surface	$1 - 4r - 2rBi_{cell}$	$r \le 1/[2(2 + Bi_{cell})]$
corner	$1 - 4r - 4rBi_{cell}$	$r \le 1/[4(1 + Bi_{cell})]$

The **corner**, cooled on two faces but storing heat in only a quarter cell, sets the limit (Incropera et al., Table 5.3):

```python
def limits(N):
    """The three positive-weight limits of r on a grid with N intervals."""
    b = h * (L / N) / k
    return {"interior": 1 / 4, "surface": 1 / (2 * (2 + b)), "corner": 1 / (4 * (1 + b))}


for n_int in (2, 10):
    d = L / n_int
    lim = limits(n_int)
    print(f"N = {n_int:2d}, dx = {d * 1000:4.1f} mm, Bi_cell = {h * d / k:.4f}:  r <= "
          + ", ".join(f"{name} {value:.4f}" for name, value in lim.items()))
    print(f"        so  dt <= {lim['corner'] * d**2 / alpha:.3f} s")
```

```
N =  2, dx = 25.0 mm, Bi_cell = 0.5208:  r <= interior 0.2500, surface 0.1983, corner 0.1644
        so  dt <= 9.397 s
N = 10, dx =  5.0 mm, Bi_cell = 0.1042:  r <= interior 0.2500, surface 0.2376, corner 0.2264
        so  dt <= 0.518 s
```

Positive weights are a **safe** rule: they keep every new value between T_∞ and T_i. The **true** stability limit is where the worst error pattern (a checkerboard) starts to grow. Power iteration on the 1D second difference finds its growth rate μ; the 2D checkerboard changes by $1 - 2r\mu$ per step, so stability needs $r \le 1/\mu$:

```python
def largest_stable_r(N):
    """The true FTCS stability limit, from the worst error pattern (power iteration)."""
    b = h * (L / N) / k
    v = [(-1.0) ** i for i in range(N + 1)]          # start from a saw-tooth
    mu = 0.0
    for _ in range(20_000):
        w = [-second_difference(v, i, N, b) for i in range(N + 1)]
        mu = max(abs(value) for value in w) / max(abs(value) for value in v)
        v = w
        scale = max(abs(value) for value in v)
        v = [value / scale for value in v]           # keep the numbers from growing
    return 1 / mu


for n_int in (2, 10):
    print(f"N = {n_int:2d}: positive weights r <= {limits(n_int)['corner']:.4f},"
          f"  true limit r = {largest_stable_r(n_int):.4f}  (interior: 0.25)")
```

```
N =  2: positive weights r <= 0.1644,  true limit r = 0.2293  (interior: 0.25)
N = 10: positive weights r <= 0.2264,  true limit r = 0.2491  (interior: 0.25)
```

On the coarse grid even $r = 0.24 < 1/4$ is unstable: the convective corner, not the interior, sets the time step.

Implicit: Crank–Nicolson and ADI. An implicit method has no stability limit. Crank–Nicolson averages the old and new time levels and is second order in time, but in 2D every step is one system of $(N+1)^2$ equations with five diagonals. **ADI** (Peaceman and Rachford, 1955) splits the step in two halves, each implicit in one direction only:

$$T^* - T^n = \tfrac{r}{2}\left(\delta_x^2 T^* + \delta_y^2 T^n\right), \qquad T^{n+1} - T^* = \tfrac{r}{2}\left(\delta_x^2 T^* + \delta_y^2 T^{n+1}\right),$$

plus the constant T_∞ terms of the cooled surfaces in each half. In the first half the unknowns of one row are coupled only to each other: a tridiagonal system for each row; in the second half, for each column. With the ghost nodes the matrix is the same for every line:

position	a_i (below)	b_i (diagonal)	c_i (above)
$i = 0$ (symmetry)	—	$1 + r$	$-r$
inside	$-r/2$	$1 + r$	$-r/2$
$i = N$ (convection)	$-r$	$1 + r\,(1 + Bi_{cell})$	—

The Thomas algorithm of Problem 1 solves each line:

```
def thomas(a, b, c, r):
    """Solve a tridiagonal system directly (Thomas algorithm)."""
    n = len(b)
    c_star = [0.0] * n
    r_star = [0.0] * n

    # forward sweep: remove the unknown before the diagonal from every equation
    c_star[0] = c[0] / b[0]
    r_star[0] = r[0] / b[0]
    for i in range(1, n):
        denominator = b[i] - a[i] * c_star[i - 1]
        c_star[i] = c[i] / denominator
        r_star[i] = (r[i] - a[i] * r_star[i - 1]) / denominator

    # back substitution
    T = [0.0] * n
    T[n - 1] = r_star[n - 1]
    for i in range(n - 2, -1, -1):
        T[i] = r_star[i] - c_star[i] * T[i + 1]

    return T
```

```
def adi_step(T, N, r, b):
    """One ADI (Peaceman-Rachford) time step on the quarter billet; returns a new grid."""
    s = r / 2
    a = [-s] * (N + 1)                    # the tridiagonal matrix, the same for every line
    diag = [1 + 2 * s] * (N + 1)
    c = [-s] * (N + 1)
    c[0] = -2 * s                         # symmetry: ghost node T[-1] = T[1]
    a[N] = -2 * s                         # convection: ghost node from the surface condition
    diag[N] = 1 + 2 * s * (1 + b)
    source = 2 * s * b * T_inf            # the T_inf term of one direction, on the surface

    # half step 1: implicit along each row j, explicit in y
    columns = [[T[j][i] for j in range(N + 1)] for i in range(N + 1)]
    half = []
    for j in range(N + 1):
        rhs = []
        for i in range(N + 1):
            value = T[j][i] + s * second_difference(columns[i], j, N, b)
            if j == N:
                value += source           # y-convection, explicit
            if i == N:
                value += source           # x-convection, implicit side's constant
            rhs.append(value)
        half.append(thomas(a, diag, c, rhs))

    # half step 2: implicit along each column i, explicit in x
    new = [[0.0] * (N + 1) for _ in range(N + 1)]
    for i in range(N + 1):
        rhs = []
        for j in range(N + 1):
            value = half[j][i] + s * second_difference(half[j], i, N, b)
            if i == N:
                value += source
            if j == N:
                value += source
            rhs.append(value)
        column = thomas(a, diag, c, rhs)
        for j in range(N + 1):
            new[j][i] = column[j]
    return new
```

ADI is unconditionally stable and second order in time, like Crank–Nicolson, at the cost of $2(N+1)$ small tridiagonal solves per step.

Step 5 · Solution setup

Initial condition $T = 600$ °C everywhere. **FTCS:** the corner rule on $N = 10$ allows $\Delta t \leq 0.518$ s; we take $\Delta t = 0.5$ s. **ADI:** we take $\Delta t = 10$ s. One function marches either method and records the centre $(0, 0)$, the middle of a face $(L, 0)$ and the corner (L, L); the time to reach 100 °C is interpolated linearly between steps:

```python
def cool(N, dt, t_end, step, snapshot_times=()):
    """March from the uniform initial temperature to t_end with the given step function."""
    dx = L / N
    r = alpha * dt / dx**2
    b = h * dx / k
    T = [[T_init] * (N + 1) for _ in range(N + 1)]
    result = {"r": r, "times": [0.0], "centre": [T_init], "surface middle": [T_init],
              "corner": [T_init], "snapshots": {}}
    for n in range(1, round(t_end / dt) + 1):
        T = step(T, N, r, b)
        t = n * dt
        result["times"].append(t)
        result["centre"].append(T[0][0])
        result["surface middle"].append(T[0][N])
        result["corner"].append(T[N][N])
        for t_snap in snapshot_times:
            if abs(t - t_snap) < 1e-6:
                result["snapshots"][t_snap] = T
    result["T"] = T
    return result


def time_to(result, T_goal):
    """Time at which the centre falls to T_goal (linear interpolation between steps)."""
    t, Tc = result["times"], result["centre"]
    for n in range(1, len(t)):
        if Tc[n] <= T_goal:
            return t[n - 1] + (Tc[n - 1] - T_goal) / (Tc[n - 1] - Tc[n]) * (t[n] - t[n - 1])
    return None
```

Step 6 · Compute

```python
t_end = 600.0
snapshots = (30.0, 120.0, 300.0, 600.0)

start = time.perf_counter()
explicit = cool(N, 0.5, t_end, ftcs_step, snapshots)
time_explicit = time.perf_counter() - start

start = time.perf_counter()
implicit = cool(N, 10.0, t_end, adi_step, snapshots)
time_implicit = time.perf_counter() - start

print(f"centre reaches {T_target:g} °C after:")
for name, res, run_time in (("FTCS, dt = 0.5 s", explicit, time_explicit),
                            ("ADI,  dt = 10 s ", implicit, time_implicit)):
    print(f"  {name}  {time_to(res, T_target):6.1f} s   (r = {res['r']:.4f},"
          f" {len(res['times']) - 1:4d} steps, run time {run_time:.3f} s)")
print(f"  exact             {t_exact:6.1f} s")
```

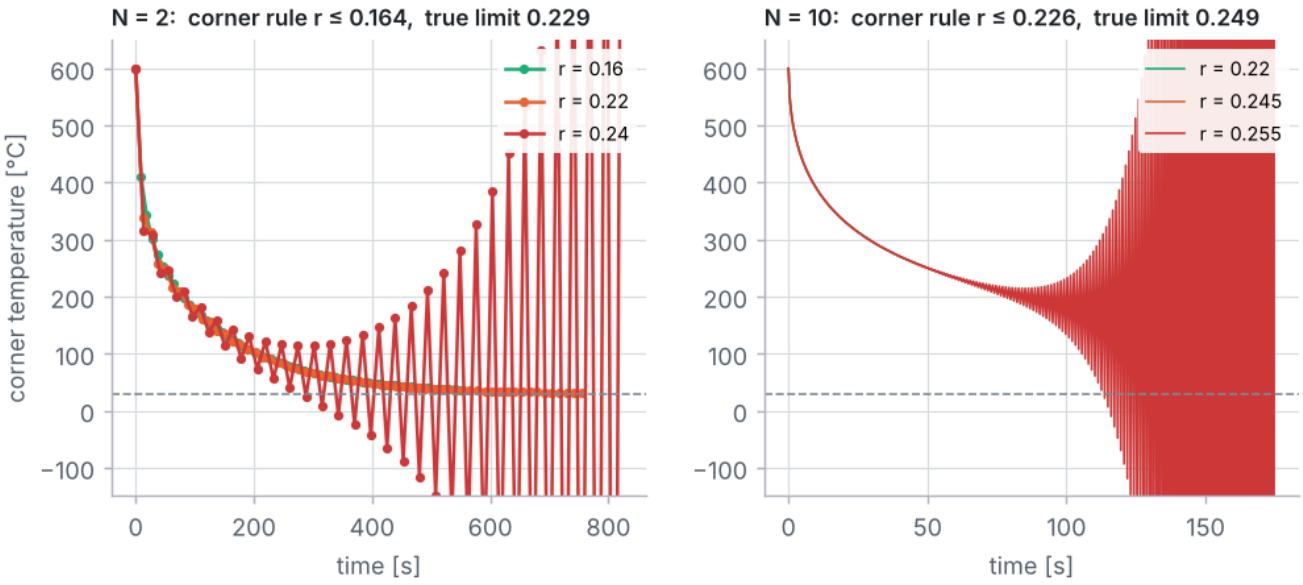
```
centre reaches 100 °C after:
  FTCS, dt = 0.5 s   348.6 s   (r = 0.2187, 1200 steps, run time 0.074 s)
  ADI,  dt = 10 s    349.2 s   (r = 4.3732,   60 steps, run time 0.009 s)
  exact             349.1 s
```

Breaking the stability limit on purpose. FTCS with three values of r on each grid:

```python
def corner_history(N, r, n_steps):
    """Run FTCS with a given r and return (times, corner temperatures)."""
    dt = r * (L / N) ** 2 / alpha
    res = cool(N, dt, n_steps * dt, ftcs_step)
    return res["times"], res["corner"]


stability_cases = {}
print("corner temperature [°C] after 1, 2, 3, 4 steps, and at the end:")
for n_int, r_values, n_steps in ((2, (0.16, 0.22, 0.24), 60), (10, (0.22, 0.245, 0.255), 300)):
    runs = {}
    for r in r_values:
        times, corner = corner_history(n_int, r, n_steps)
        runs[f"r = {r}"] = (times, corner)
        print(f"N = {n_int:2d}, r = {r:5.3f}, dt = {times[1]:6.3f} s: "
              + " ".join(f"{value:6.1f}" for value in corner[1:5])
              + f"  ... {corner[-1]:8.4g} °C after {n_steps} steps")
    title = (f"N = {n_int}:  corner rule r ≤ {limits(n_int)['corner']:.3f},"
             f"  true limit {largest_stable_r(n_int):.3f}")
    stability_cases[title] = runs
```

```
corner temperature [°C] after 1, 2, 3, 4 steps, and at the end:
N =  2, r = 0.160, dt =  9.147 s:  410.0  344.1  301.4  274.4  ...    36.57 °C after 60 steps
N =  2, r = 0.220, dt = 12.577 s:  338.8  312.2  257.9  247.4  ...    31.52 °C after 60 steps
N =  2, r = 0.240, dt = 13.720 s:  315.0  309.3  242.2  246.0  ...     1499 °C after 60 steps
N = 10, r = 0.220, dt =  0.503 s:  547.8  523.3  505.7  492.2  ...    138.1 °C after 300 steps
N = 10, r = 0.245, dt =  0.560 s:  541.8  518.1  500.0  486.3  ...    126.4 °C after 300 steps
N = 10, r = 0.255, dt =  0.583 s:  539.4  516.2  497.8  484.1  ... 1.94e+04 °C after 300 steps
```



Corner temperature for three values of r. Below the corner rule: smooth cooling. Between the two limits: stable, but a staircase on the coarse grid. Above the true limit: growing oscillations (the axis is clipped).

The whiteboard numbers. The whiteboard guide does two FTCS steps ($N = 2$, $r = 0.14$) and one ADI step five times as long ($r = 0.7$) by hand:

```
def show(T, title):
    """Print a grid row by row, from the top down."""
    print(title)
    for j in range(len(T) - 1, -1, -1):
        print(f"   j = {j}: " + "  ".join(f"{value:8.3f}" for value in T[j]))


N_hand = 2
dx_hand = L / N_hand
b_hand = h * dx_hand / k
r_hand = 0.14
dt_hand = r_hand * dx_hand**2 / alpha
print(f"N = {N_hand}: dx = {dx_hand * 1000:g} mm, Bi_cell = {b_hand:.6f}, r = {r_hand},"
      f" dt = {dt_hand:.4f} s\n")

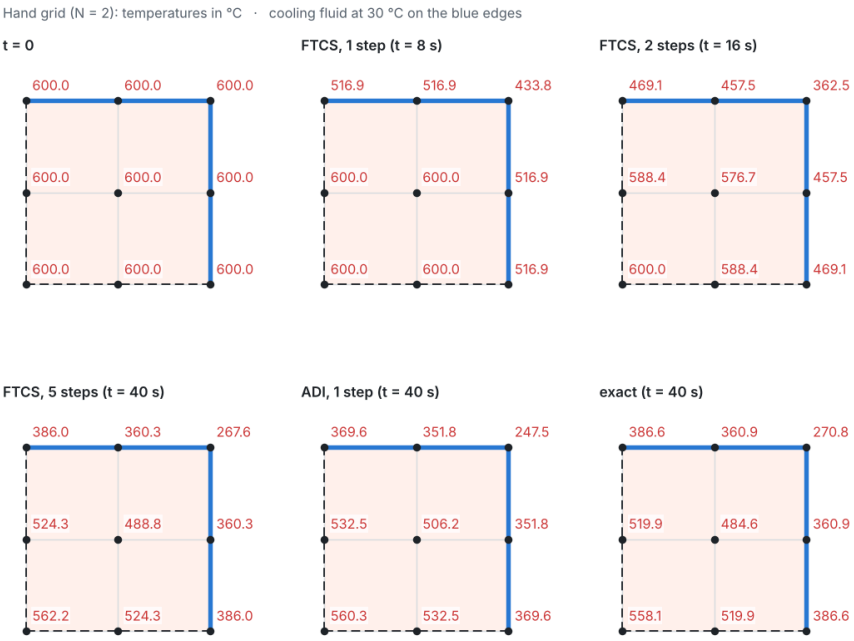
T_hand = [[T_init] * (N_hand + 1) for _ in range(N_hand + 1)]
hand_states = {"t = 0": T_hand}
for n in (1, 2):
    T_hand = ftcs_step(T_hand, N_hand, r_hand, b_hand)
    hand_states[f"FTCS, {n} step{'s' if n > 1 else ''} (t = {n * dt_hand:.0f} s)"] = T_hand
    show(T_hand, f"FTCS after {n} step(s), t = {n * dt_hand:.2f} s:")

T_ftcs5 = [[T_init] * (N_hand + 1) for _ in range(N_hand + 1)]
for n in range(5):
    T_ftcs5 = ftcs_step(T_ftcs5, N_hand, r_hand, b_hand)
T_adi = adi_step([[T_init] * (N_hand + 1) for _ in range(N_hand + 1)], N_hand, 5 * r_hand, b_hand)
T_ex_hand = [[T_exact(i * dx_hand, j * dx_hand, 5 * dt_hand) for i in range(N_hand + 1)]
             for j in range(N_hand + 1)]
hand_states[f"FTCS, 5 steps (t = {5 * dt_hand:.0f} s)"] = T_ftcs5
hand_states[f"ADI, 1 step (t = {5 * dt_hand:.0f} s)"] = T_adi
hand_states[f"exact (t = {5 * dt_hand:.0f} s)"] = T_ex_hand
show(T_ftcs5, f"\nFTCS after 5 steps, t = {5 * dt_hand:.2f} s:")
show(T_adi, f"ADI after 1 step of {5 * dt_hand:.2f} s (r = {5 * r_hand:.2f}):")
show(T_ex_hand, f"exact solution at the nodes, t = {5 * dt_hand:.2f} s:")
```

```
N = 2: dx = 25 mm, Bi_cell = 0.520833, r = 0.14, dt = 8.0033 s

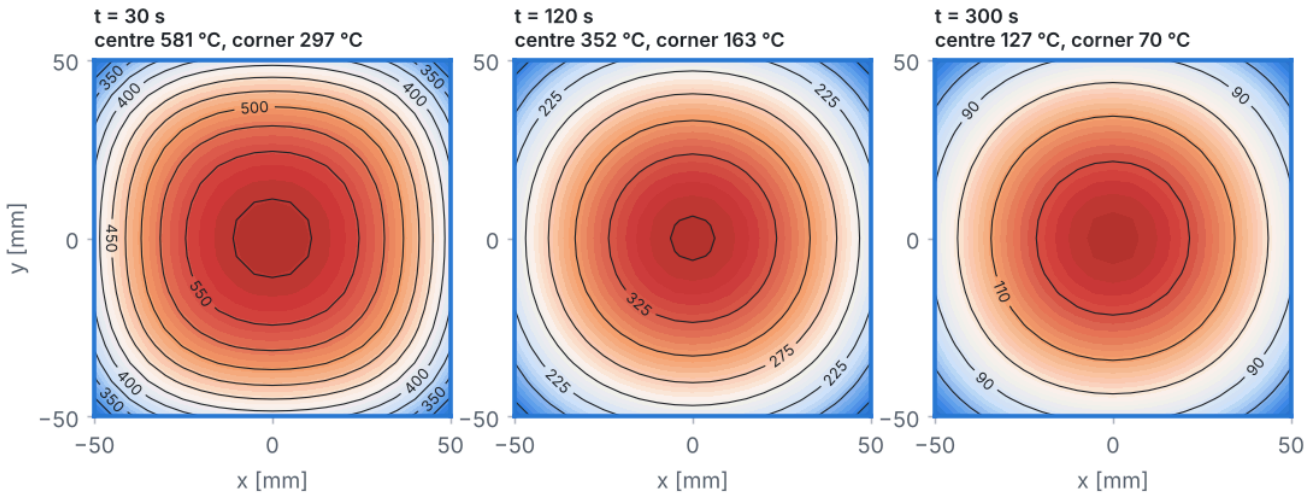
FTCS after 1 step(s), t = 8.00 s:
   j = 2:  516.875   516.875   433.750
   j = 1:  600.000   600.000   516.875
   j = 0:  600.000   600.000   516.875
FTCS after 2 step(s), t = 16.01 s:
   j = 2:  469.147   457.510   362.540
   j = 1:  588.362   576.725   457.510
   j = 0:  600.000   588.362   469.147

FTCS after 5 steps, t = 40.02 s:
   j = 2:  385.993   360.276   267.631
   j = 1:  524.260   488.790   360.276
   j = 0:  562.174   524.260   385.993
ADI after 1 step of 40.02 s (r = 0.70):
   j = 2:  369.598   351.786   247.457
   j = 1:  532.525   506.167   351.786
   j = 0:  560.342   532.525   369.598
exact solution at the nodes, t = 40.02 s:
   j = 2:  386.611   360.859   270.822
   j = 1:  519.939   484.559   360.859
   j = 0:  558.072   519.939   386.611
```

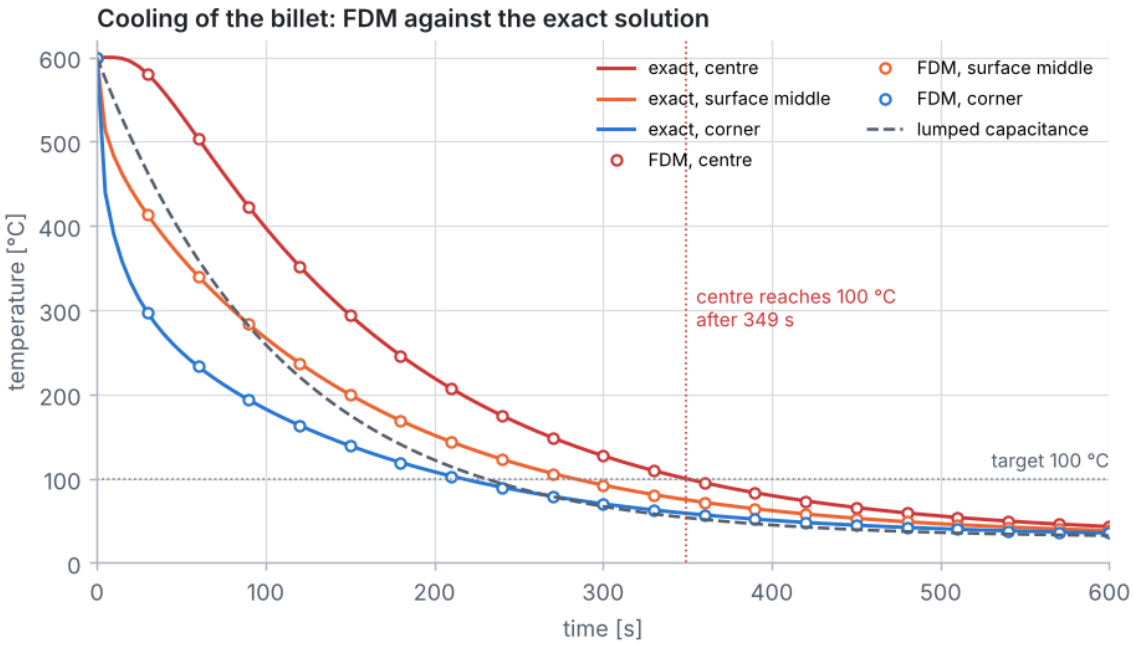


The hand grid: two FTCS steps, then FTCS after five steps, one ADI step of the same length, and the exact solution, all at $t = 40$ s.

Step 7 · Post-processing and verification



Temperature over the whole cross-section (the quarter mirrored), each map with its own colour scale. The isotherms are rounded squares; the corners are always coldest.



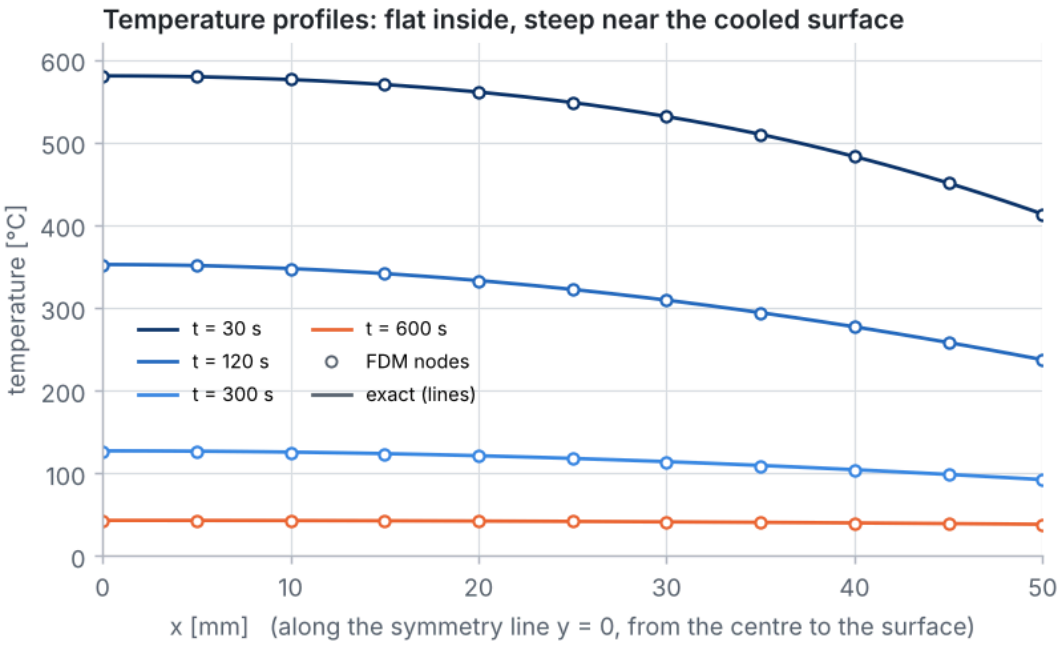
Temperature against time at the centre, the middle of a face and the corner (FTCS points, exact lines), and the lumped-capacitance prediction.

The centre does not notice the cooling for the first half minute, then lags far behind the surface. Lumped capacitance is too cold for the centre and too warm for the surface; it predicts 100 °C after 230 s, a third too early.

```
profile_times = (30.0, 120.0, 300.0, 600.0)
x_mm = [1000 * xi for xi in x]
numerical_profiles = {t: explicit["snapshots"][t][0] for t in profile_times}  # row j = 0: y = 0
x_dense = [L * m / 100 for m in range(101)]
exact_profiles = {t: ([1000 * xi for xi in x_dense], [T_exact(xi, 0, t) for xi in x_dense])
                  for t in profile_times}
plots.profiles(x_mm, numerical_profiles, exact_profiles)

print("x [mm]" + "".join(f"     t = {t:3.0f} s     " for t in profile_times))
print("      " + "     FDM    exact" * len(profile_times))
for i in range(0, N + 1, 2):
    print(f"{x_mm[i]:5.1f} " + "".join(f"  {numerical_profiles[t][i]:7.2f} {T_exact(x[i], 0, t):7.2f}"
                                       for t in profile_times))
```

```
x [mm]     t =  30 s         t = 120 s         t = 300 s         t = 600 s
           FDM    exact      FDM    exact      FDM    exact      FDM    exact
  0.0    580.73  580.39   352.11  352.36   126.84  127.14    43.05   43.14
 10.0    576.21  575.87   347.23  347.49   125.36  125.67    42.85   42.94
 20.0    561.09  560.79   332.74  333.00   120.99  121.28    42.26   42.35
 30.0    531.46  531.25   309.04  309.31   113.84  114.11    41.30   41.38
 40.0    482.94  482.86   276.82  277.09   104.14  104.39    39.99   40.06
 50.0    413.28  413.35   237.03  237.30    92.18   92.41    38.38   38.44
```



Temperature along the symmetry line $y = 0$: flat inside, steep near the cooled surface.

Cost against accuracy (all on $N = 10$; run times depend on the computer):

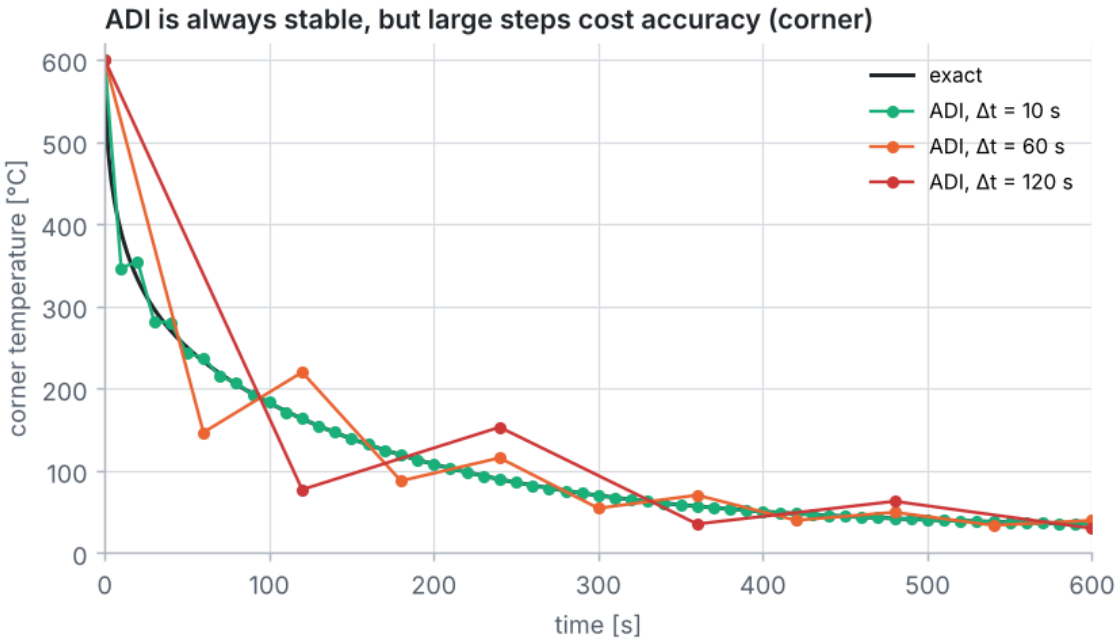
```
runs = [("FTCS", ftcs_step, 0.5)] + [("ADI", adi_step, dt) for dt in (5.0, 10.0, 20.0, 30.0, 60.0)]
T_exact_300 = T_exact(0, 0, 300.0)
comparison = []
print("                                          centre at 300 s      reaches 100 °C")
print("method  dt [s]      r   steps  time [s]   T [°C]    error      t [s]   error")
for name, step, dt in runs:
    start = time.perf_counter()
    res = cool(N, dt, t_end, step)
    run_time = time.perf_counter() - start
    T300 = res["centre"][round(300 / dt)]
    t_goal = time_to(res, T_target)
    comparison.append({"method": name, "dt": dt, "r": res["r"], "steps": len(res["times"]) - 1,
                       "T_centre_300": T300, "time_to_target": t_goal})
    print(f"{name:6s} {dt:6.1f} {res['r']:7.3f} {len(res['times']) - 1:6d} {run_time:8.3f}"
          f"  {T300:8.3f}  {T300 - T_exact_300:+7.3f}   {t_goal:7.1f}  {t_goal - t_exact:+5.1f}")
print(f"exact                                    {T_exact_300:8.3f}             {t_exact:7.1f}")
```

```
                                    centre at 300 s      reaches 100 °C
method  dt [s]      r   steps  time [s]   T [°C]    error      t [s]   error
FTCS       0.5   0.219   1200    0.057   126.838   -0.306     348.6   -0.6
ADI        5.0   2.187    120    0.015   127.158   +0.014     349.2   +0.0
ADI       10.0   4.373     60    0.008   127.144   +0.000     349.2   +0.0
ADI       20.0   8.746     30    0.004   127.091   -0.053     349.4   +0.2
ADI       30.0  13.120     20    0.003   127.001   -0.143     349.6   +0.4
ADI       60.0  26.239     10    0.001   125.507   -1.637     350.1   +0.9
exact                                    127.144              349.1
```

- ADI with $\Delta t = 5$ s is about 20 times more accurate at the centre than FTCS with $\Delta t = 0.5$ s, and several times cheaper. FTCS is first order in time, and on this grid its error comes almost entirely from Δt.
- The tiny ADI errors at $\Delta t = 5$ and 10 s are partly luck: the spatial error of the grid ($+0.018$ °C at the centre) and the time error have opposite signs.
- With $\Delta t = 60$ s the centre is 1.6 °C too cold at 300 s. Stability is not accuracy.

```
large_runs = {}
print("corner temperature [°C] after 1, 2, 3, 4 steps")
for dt in (10.0, 60.0, 120.0):
    res = cool(N, dt, t_end, adi_step)
    large_runs[f"ADI, Δt = {dt:g} s"] = (res["times"], res["corner"])
    print(f"dt = {dt:5.1f} s   ADI: " + " ".join(f"{value:6.1f}" for value in res["corner"][1:5])
          + "    exact: " + " ".join(f"{T_exact(L, L, n * dt):6.1f}" for n in range(1, 5)))
t_dense = [0.0] + [2.0 * n for n in range(1, 301)]
plots.adi_large_steps(large_runs, (t_dense, [T_exact(L, L, t) for t in t_dense]), "corner")
```

```
corner temperature [°C] after 1, 2, 3, 4 steps
dt =  10.0 s   ADI:  346.6  354.9  282.1  280.1    exact:  390.1  333.0  297.0  270.9
dt =  60.0 s   ADI:  146.8  220.0   87.8  115.8    exact:  233.2  163.3  119.2   89.8
dt = 120.0 s   ADI:   77.2  153.1   35.4   63.0    exact:  163.3   89.8   56.9   42.1
```



ADI with large steps: the corner temperature oscillates (it even rises, which is impossible for a cooling billet) before the oscillation dies out.

ADI, like Crank–Nicolson, damps the fastest-changing patterns only weakly when r is large, so the sudden start of cooling at the corner rings for a few steps. The time step must still be chosen for accuracy: small at the start, larger later. Commercial codes adapt their time step automatically for this reason.

Verification 1: refining the grid. FTCS with $r \approx 0.2$, so Δt shrinks with Δx^2, against the exact solution at $t = 300$ s:

```
grid_study = {"dx": [], "max_error": [], "centre_error": []}
print("   N  dx [mm]   dt [s]   steps  time [s]   largest error [°C]  ratio  centre error [°C]")
for n_int in (5, 10, 20, 40):
    d = L / n_int
    n_steps = math.ceil(300 / (0.2 * d**2 / alpha))     # r close to 0.2, ending exactly at 300 s
    dt = 300 / n_steps
    start = time.perf_counter()
    res = cool(n_int, dt, 300.0, ftcs_step)
    run_time = time.perf_counter() - start
    error = max(abs(res["T"][j][i] - T_exact(i * d, j * d, 300.0))
                for j in range(n_int + 1) for i in range(n_int + 1))
    ratio = f"{grid_study['max_error'][-1] / error:5.2f}" if grid_study["max_error"] else "     "
    grid_study["dx"].append(d * 1000)
    grid_study["max_error"].append(error)
    grid_study["centre_error"].append(res["T"][0][0] - T_exact(0, 0, 300.0))
    print(f"{n_int:4d}  {d * 1000:6.2f}  {dt:7.4f}  {n_steps:6d}  {run_time:8.2f}"
          f"   {error:14.5f}     {ratio}   {grid_study['centre_error'][-1]:+12.5f}")
```

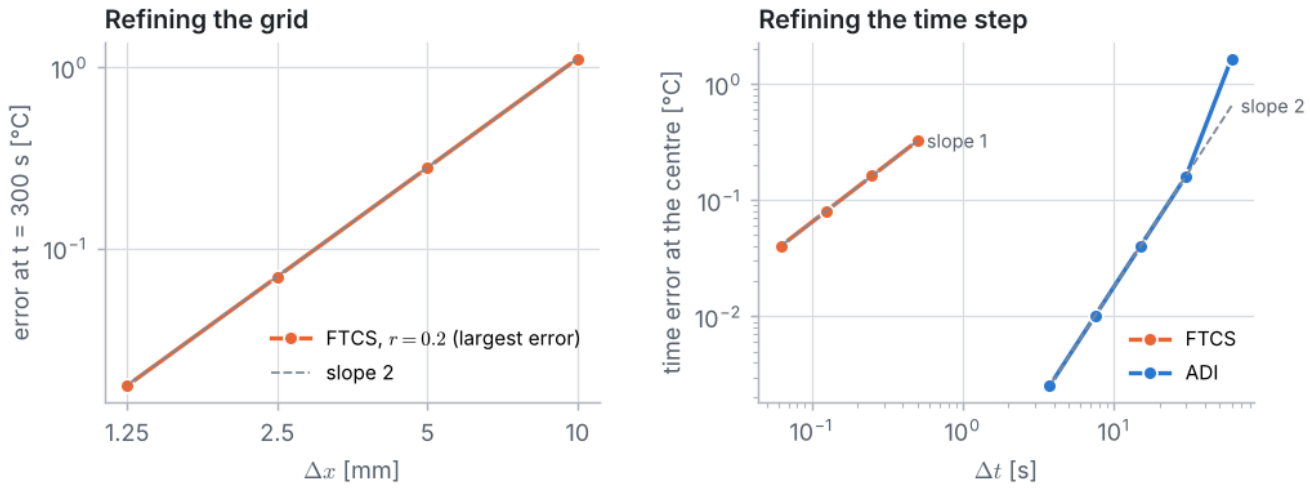
```
   N  dx [mm]   dt [s]   steps  time [s]   largest error [°C]  ratio  centre error [°C]
   5   10.00   1.8293     164      0.00          1.11765                -1.11765
  10    5.00   0.4573     656      0.06          0.27848      4.01      -0.27848
  20    2.50   0.1143    2624      0.49          0.06956      4.00      -0.06956
  40    1.25   0.0286   10496      6.83          0.01739      4.00      -0.01739
```

The error drops by 4 each time Δx is halved: **second order**. Each halving multiplies the FTCS work by 16.

Verification 2: refining the time step. On the same grid ($N = 10$), against a run with a tiny time step, so that only the time error remains:

```
reference = cool(N, 0.05, 300.0, adi_step)["centre"][-1]
time_study = {"FTCS": ([], []), "ADI": ([], [])}
print(f"reference (same grid, ADI dt = 0.05 s): centre at 300 s = {reference:.5f} °C")
print("method   dt [s]    time error [°C]   ratio")
for name, step, dts in (("FTCS", ftcs_step, (0.5, 0.25, 0.125, 0.0625)),
                        ("ADI", adi_step, (60.0, 30.0, 15.0, 7.5, 3.75))):
    for dt in dts:
        error = abs(cool(N, dt, 300.0, step)["centre"][-1] - reference)
        errs = time_study[name][1]
        ratio = f"{errs[-1] / error:5.2f}" if errs else ""
        time_study[name][0].append(dt)
        errs.append(error)
        print(f"{name:6s}  {dt:7.4f}       {error:10.6f}      {ratio}")
```

```
reference (same grid, ADI dt = 0.05 s): centre at 300 s = 127.16214 °C
method   dt [s]    time error [°C]   ratio
FTCS     0.5000         0.324405
FTCS     0.2500         0.162157       2.00
FTCS     0.1250         0.081067       2.00
FTCS     0.0625         0.040531       2.00
ADI     60.0000         1.654798
ADI     30.0000         0.161481      10.25
ADI     15.0000         0.040478       3.99
ADI      7.5000         0.010135       3.99
ADI      3.7500         0.002533       4.00
```



left: FDM against the exact solution (Δt shrinks with Δx²). right: same grid (N = 10), against a run with a tiny time step.

Left: error against the exact solution on four grids (slope 2). Right: time error on one grid, FTCS first order and ADI second order.

Result. The centre of the billet reaches 100 °C after **349 s** (about 5.8 minutes): exact 349.1 s, FTCS ($N = 10$, $\Delta t = 0.5$ s) 348.6 s, ADI ($N = 10$, $\Delta t = 10$ s) 349.2 s. At that moment the middle of a face is at 75 °C and the corner at 59 °C. Lumped capacitance ($Bi = 0.52$) would have predicted 230 s, 34 % too early. FTCS needs $\Delta t \leq 0.518$ s because of the cooled corner; ADI is stable for any Δt, second order in time, and with $\Delta t = 5$ to 10 s more accurate than FTCS at a fraction of the cost.

Key ideas

- $Bi = hL_c/k > 0.1$: lumped capacitance fails, and a temperature field is needed.
- Symmetry reduces the billet to a quarter; ghost nodes on the symmetry lines, energy balances on half and quarter cells at the cooled surfaces.
- Every node equation is a sum of an x and a y second difference; this is what makes ADI possible.
- The explicit time step is limited by the cooled corner: $r \leq 1/[4(1 + Bi_{cell})]$ is safe; the true limit lies between this and 1/4.
- ADI = two sets of tridiagonal solves (Thomas algorithm) per step: unconditionally stable, second order in time, but large steps still cost accuracy.
- The exact product solution verifies the code: second order in Δx; first (FTCS) and second (ADI) order in Δt.